

Photo-z estimate

Lorenzo Lavacchini - 7171024

Introduzione

Il principale obiettivo scientifico alla base degli esperimenti cosmologici attuali e futuri è l'esplorazione del settore oscuro (dark sector).

Studi come Euclid ed LSST perseguiranno questo obiettivo andando a fare una mappatura dell'universo in maniera estesa e senza precedenti.

Il tutto però, è possibile solo tramite misurazioni degli oggetti cosmici osservabili, misure che richiedono alla base la necessità di stimare il *redshift*.

Prima di procedere ulteriormente è interessante ed utile ricordare il significato fisico del parametro di redshift. Letteralmente significa "spostamento verso il rosso", e rappresenta una stima della velocità di allontanamento di un oggetto cosmico che emetta o assorba luce elettromagnetica, come ad esempio una galassia. È una quantità fondamentale, poiché permette di ricavare distanza (tramite la legge di Hubble) e l'età degli oggetti cosmici analizzati.

Veniamo ora al problema della stima del redshift. Ci sono due approcci principali per stimarlo.

- **spectroscopic observations:**

è il metodo più preciso, ma costoso in termini di tempo di osservazione, e difficoltoso per oggetti con redshift alto.

Inoltre, funziona bene solo per stimare il redshift per oggetti singoli, ma non per oggetti multipli contemporaneamente.

- **photometric redshifts (photo-z):**

è un metodo meno preciso, ma meno costoso e permette di stimare il redshift per oggetti multipli contemporaneamente.

L'approccio ritenuto più vantaggioso ed utilizzato nel paper di riferimento è il **photometric redshift**, approccio in cui la quantità di redshift viene denominata **photo-z**.

Il photometric redshift, come evidente dal nome, si basa sulla fotometria. Quest'ultima analizza l'intensità luminosa di un oggetto osservato attraverso vari filtri standard, ognuno dei quali facente passare una banda passante abbastanza ampia ma specifica, come ad esempio la luce rossa, la luce verde, o la luce blu. Utilizzando queste misurazioni, si può stimare il redshift utilizzando vari modelli, che possono ad esempio essere statistici, o di ML.

Obiettivo del lavoro

In questo lavoro, l'obiettivo è quello di andare a confrontare vari modelli di ML per stimare il parametro di redshift, all'interno dell'approccio di **photometric redshift**.

I vari modelli che andrò ad implementare e a confrontare sono i seguenti:

- Baseline (basato sulla media del valore target del training set)
- Linear Regression

- Decision Tree
- Explainable Boosting Machines
- Heteroskedastic Likelihood and Multi-Latent GP

Il primo “modello” è una baseline banale, in cui si va a predire utilizzando la media dei targets di train. Serve soltanto come punto di partenza da cui partire a confrontare le prestazioni degli altri modelli.

I successivi tre modelli sono standard, rientrano nella categoria dei modelli interpretabili by design, e servono come baseline interpretabile per il problema.

Il quarto metodo non è considerabile di per sé come interpretabile, ma ha una caratteristica utile che rende sensato prenderlo in considerazione. I GP infatti non restituiscono solo una previsione puntuale, ma un'intera distribuzione di probabilità. Questo permette di sapere non solo *cosa* il modello prevede, ma anche quanto è incerto su quella previsione (intervalli di confidenza).

Il quarto metodo non è considerabile di per sé come interpretabile, ma dato che è innovativo, è interessante valutarne l'applicazione a questo tipo di problema. Esso è strutturato in modo tale da consentire ottime prestazioni in presenza di dati eteroschedastici. Sarà quindi analizzata la struttura dei dataset per verificare la possibilità di applicazione del modello a questo caso di studio, e poi, eventualmente, saranno analizzate sia l'implementazione, sia la performance del modello.

L'analisi di questi modelli sarà fatta in maniera indipendente per ciascuno dei quattro dataset a disposizione, ciascuno con un differente grado di covariate shift. Nell'analisi, andrò a confrontare le performance dei vari modelli sui dati, valutandone l'eventuale (e prevedibile) calo a mano a mano che il grado di covarianza shift aumenta.

Un secondo obiettivo del lavoro è quello di fornire, tramite tecniche e modelli utilizzati, risultati interpretabili sul ruolo delle features nelle predizioni eseguite.

Problema nell'uso di modelli ML per predire il redshift

Il problema principale deriva dal cosiddetto *Covariate Shift*.

Esso si verifica quando la distribuzione delle variabili indipendenti (i covariati, o insomma, le features) differiscono tra il training set, ed il test set (o comunque il set di dati a cui si vuole applicare il modello).

In astronomia, questo problema è una conseguenza dovuta al bias di selezione. In pratica, i campioni di dati (usati come training set) non sono campioni casuali della popolazione totale osservata dagli studi fotometrici, ma sono condizionati da un bias.

Un esempio di bias di selezione in questo campo di studi è il *Malquist Bias*, in cui man mano che aumenta la distanza degli oggetti luminosi in esame, si osservano sistematicamente di più gli oggetti più luminosi. Questo si verifica poiché ottenere lo spettro di oggetti deboli richiede tempi di osservazione estremamente lunghi e proibitivi, cosa che porta ad avere come conseguenza un training set con bias verso sorgenti più luminose, e con redshift inferiori.

L'esistenza di covariate shift è un problema, perché impedisce ai modelli addestrati di generalizzare bene.

Da un punto di vista matematico, il covariate shift nel caso della predizione di photo-z è definito da due condizioni, ovvero:

- La distribuzione delle features nel training set (source) è diversa dalla distribuzione delle features del test set (target):

$$p_S(x) \neq p_T(x)$$

cosa che porta quindi a:

$$p_S(x, z) \neq p_T(x, z)$$

- La relazione tra le caratteristiche fotometriche e il redshift (la distribuzione condizionata $p(z | x)$) rimane la stessa per entrambi i gruppi. In realtà questa è un'assunzione che si fa.

$$p_S(z|x) = p_T(z|x)$$

I dataset utilizzati

I dataset disponibili sono quattro, ciascuno con un grado di covariate shift diverso presente tra training set e test set.

In particolare, i gradi di covariate shift che contraddistinguono i dataset sono:

- **noCS** : assenza di covariate shift
- **weakCS** : covariate shift debole
- **mildCS** : covariate shift leggero
- **strongCS** : covariate shift forte

Come detto, ciascun modello sarà analizzato su ciascun dataset, per poi fare un confronto globale dei risultati ottenuti.

Più in dettaglio, ciascuno dei training set consiste di N_S galassie campionate dalla distribuzione $P_S(x, z)$ mentre ciascun target dataset consiste di N_T galassie campionate dalla distribuzione $p_T(x, z)$.

Si noti come S stia per "Source" e T per "Target", ma altro non sono che training set e test set.

A questo punto, chiamando:

- $z^{(i)}$ il redshift (la label)
- $x^{(i)}$ una generica feature (che sarà un colore o la magnitudine fotometrica)

allora si può esprimere i due dataset in questo modo:

- $D_S = \{x_S^{(i)}, z_S^{(i)}\}_{i=1}^{N_S}$
- $D_T = \{x_T^{(i)}, z_T^{(i)}\}_{i=1}^{N_T}$

Analizziamo adesso le features utilizzate nei dataset. Esse sono di due tipi:

- **Magnitudini**: indicano quanto un oggetto appare luminoso in un filtro specifico, cioè ad una certa frequenza specifica.
- **Colore**: rappresentano la differenza di magnitudine tra due bande adiacenti.

Nei dataset utilizzati, in particolare abbiamo:

- **r** (magnitudine) : magnitudine del filtro rosso.
Misura cioè quanto è luminosa la sorgente in quella banda di frequenza.
- **ug** (colore) : differenza tra banda ultravioletto (u) e verde (g)
Cioè matematicamente, $ug = mag_u - mag_g$
- **gr** (colore) : differenza tra la banda verde (g) e rossa (r)

Cioè matematicamente, $gr = mag_g - mag_r$

- **ri** (colore) : differenza tra banda "vicino all'infrarosso i" (i) e rossa (r) :

Cioè matematicamente, $ri = mag_r - mag_i$

- **iz** (colore) : differenza tra banda "vicino all'infrarosso i" (i) e banda "vicina all'infrarosso z" (z):

Cioè matematicamente, $iz = mag_i - mag_z$

- **zy** (colore) : differenza tra banda "vicino all'infrarosso z" e banda "vicino all'infrarosso y" (y):

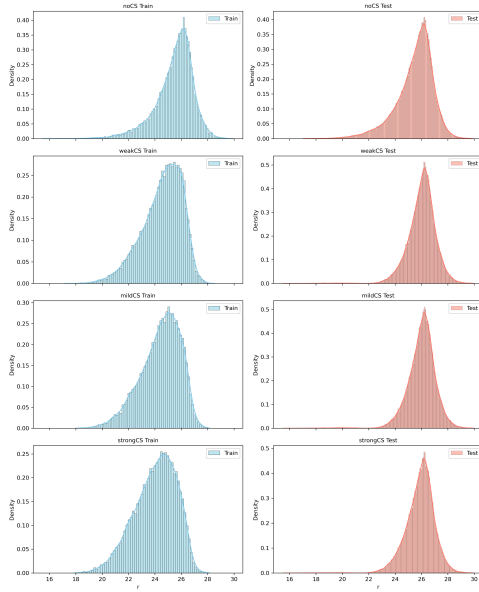
Cioè matematicamente, $zy = mag_z - mag_y$

Visualizzazione del grado di covariate shift per ciascuna feature

Di seguito è utile visualizzare graficamente, per ciascuna feature, la differenza tra distribuzione nel training set e nel test set. Questo lo si fa inoltre per ciascuno dei 4 datasets con gradi diversi di CS.

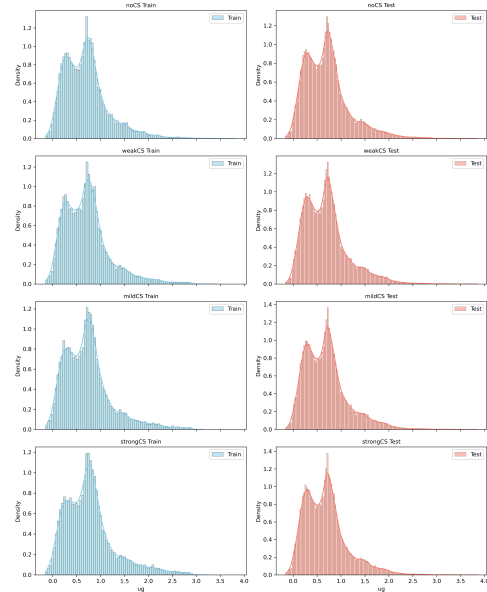
Feature **r**

Distribuzione della Feature: r attraverso i livelli di CS (Train vs Test)



Feature **ug**

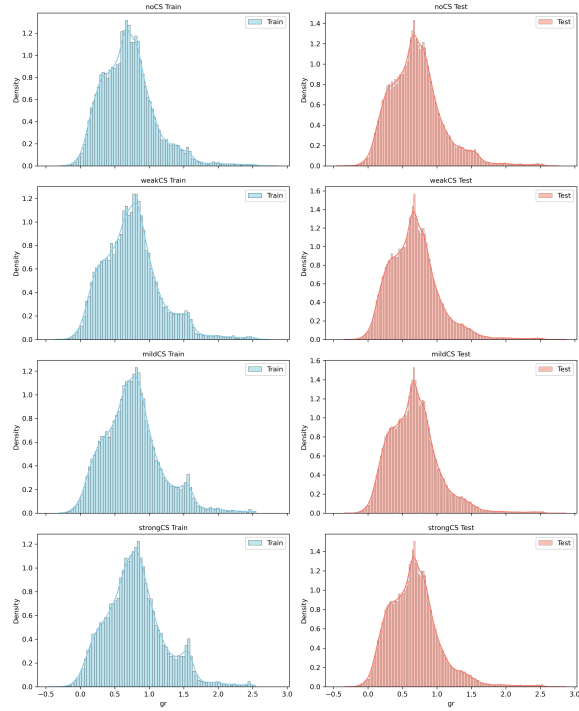
Distribuzione della Feature: ug attraverso i livelli di CS (Train vs Test)



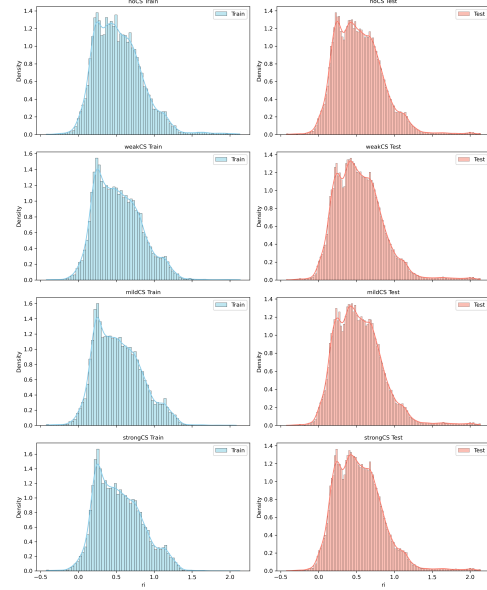
Feature **gr**

Feature **ri**

Distribuzione della Feature: gr attraverso i livelli di CS (Train vs Test)

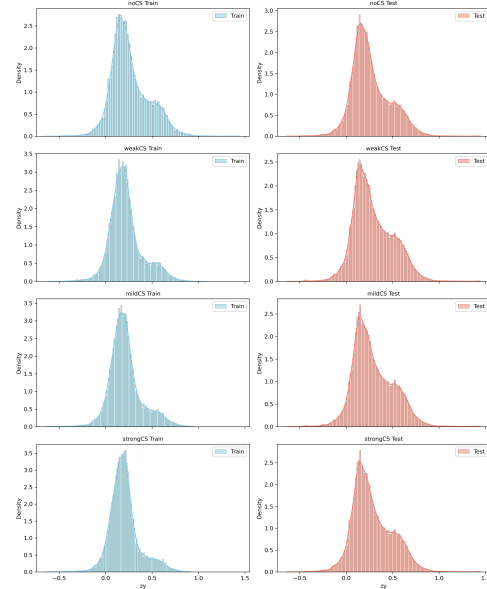


Distribuzione della Feature: ri attraverso i livelli di CS (Train vs Test)



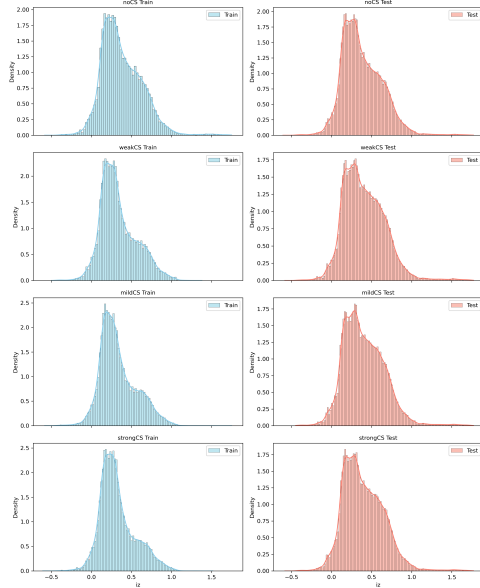
Feature **zy**

Distribuzione della Feature: zy attraverso i livelli di CS (Train vs Test)



Feature **iz**

Distribuzione della Feature: iz attraverso i livelli di CS (Train vs Test)



In effetti, si osserva come le distribuzioni delle features si differenzino sempre di più, via via che aumenta il grado di CS nel dataset. Questo ammonisce sul fatto che le prestazioni dei modelli che utilizzeremo dovrebbero tendere a peggiorare nel momento in cui aumentiamo il grado di CS presente.

L'obiettivo sarà dunque quello di individuare il modello (tra quelli che prenderemo in considerazione) che mantenga le sue prestazioni più stabili possibili all'aumentare di CS.

Specifiche della piattaforma utilizzata

Il codice per eseguire gli esperimenti è stato scritto in Python (interprete v3.12.13) utilizzando l'ambiente di sviluppo "Jupyter Lab".

L'ambiente di esecuzione è stato anch'esso Jupyter, utilizzando *nbconvert* per lanciare l'esecuzione da terminale bash.

L'hardware su cui è stato eseguito è quello di papavero, messo a disposizione dal dinfo dell'università, e connesso tramite SSH. Esso prevede 64 CPU Intel(R) Xeon(R) Silver 4314 CPU @ 2.40GHz, oltre a 62 GB di RAM. Per quanto riguarda la GPU, prevede inoltre 2 NVIDIA RTX A2000 da 12 GB.

Il sistema operativo infine installato su papavero ed utilizzato per l'esecuzione è Ubuntu 22.04.4 LTS.

Librerie utilizzate

Per l'esecuzione del notebook ho installato le seguenti librerie:

- numpy
- pandas
- matplotlib
- seaborn
- scikit-learn
- statmodels
- tqdm
- interpret
- gpflow (installa automaticamente tensorflow e tensorflow_probability come dipendenze)
- tf_keras
- kaleido (necessario per far funzionare plotly, e salvare i grafici per le EBM)
- shap

Nota: per il funzionamento corretto del notebook è stato necessario prima di tutto fare un downgrade della librerie *setuptools* ad una versione ≤ 81 . Downgrade effettuato con conda.

Nota: per far funzionare kaleido è stato necessario installarlo nella versione 0.2.1, per evitare le ultime versioni con dipendenza da librerie di Google Chrome.

Esperimenti per i modelli interpretable by design

Lo svolgimento degli esperimenti si articola nelle seguenti fasi.

Per ciascun modello da testare ed analizzare:

- Si parte istanziando il modello.

- Se il modello contiene degli iperparametri, si utilizza k-fold-cross-validation per trovare quelli migliori. Se il modello lo permette (cioè implementa l'interfaccia di scikit-learn) per fare questa operazione, si utilizzano Grid-Search o Randomized-Search, a seconda della complessità di addestramento del modello. Altrimenti, si utilizzano approcci manuali.
- Si ottiene così il modello con i migliori iperparametri (tra quelli inseriti nello spazio di ricerca).
- Si svolge una ulteriore k-fold-cross-validation per accertarsi che il modello non faccia overfitting.
Sebbene infatti usare Grid-Search o Randomized-Search ci tuteli nella scelta dei migliori iperparametri (dunque quelli che portano a migliore generalizzazione) tra quelli inseriti nello spazio di ricerca, non abbiamo certezza di avere inserito nello spazio di ricerca iperparametri idonei a non commettere overfitting (visto che per esigenze computazionali non si fa una ispezione completa di tutto lo spazio di tutti gli iperparametri, ma soltanto dei valori più significativi e promettenti).
- Si analizzano le performances medie su train set e su validation set durante la k-fold-cross-validation, per visualizzare lo scarto che c'è nelle capacità di predizione su dati di addestramento, e dati nuovi. Insomma, si valuta così la presenza o meno di overfitting.
Se si rileva overfitting, si modificano gli iperparametri o si applicano tecniche di regolarizzazione, a seconda del caso e del modello interessato
- Si riaddestra il modello su tutto il training set.
- Si valutano le performances di generalizzazione del modello sul test set, e si salvano i risultati.

Questa pipeline infine verrà eseguita inoltre per ciascuno dei 4 dataset a disposizione, così da poter verificare per ciascuno le capacità del modello analizzato.

Esperimenti per il modello GP Flow

Qui gli esperimenti hanno un procedimento diverso. Questo è dovuto alle modalità di addestramento del modello, che viene fatto iterativamente su varie epoche.

Lo svolgimento degli esperimenti si articola nelle seguenti fasi.

- Si parte istanziando il modello.
- Si fa il tuning dei due iperparametri principali, ovvero il numero di inducing points (M) e il numero di epoche.

Per farlo, si addestra ogni volta il modello con un valore di M diverso, e per ogni M si fa poi un addestramento con early stopping con patience pari a 100 per trovare il numero di epoche ottimale.

L'early stopping viene eseguito utilizzando la **Negative ELBO (Evidence Lower Bound)** sul validation set, e viene utilizzato per non fare overfitting. Una volta individuata la condizione per fare early stopping, si ripristinano i pesi del modello migliori trovati fino a quel momento (cioè quelli relativi all'ultimo miglioramento trovato della validation loss).

- Si valuta il modello sul validation set (generato da un singolo split del training set) e si salvano i risultati
- Si sceglie la configurazione del modello migliore in base all'MSE ottenuto, e se ne valutano le performances sul test set, così da confrontarlo con i modelli interpretabili precedentemente valutati.

Questa pipeline infine verrà eseguita inoltre per ciascuno dei 4 dataset a disposizione, così da poter verificare per ciascuno le capacità del modello analizzato.

Prima fase - implementazione di baseline banale e modelli interpretabili by design

Baseline banale

La baseline banale utilizza come predizione la media dei target del training set.

È dunque un modello banale, ma utile per valutare il punto di partenza del dataset e quanto sia il margine di miglioramento.

Linear Regression

La regressione Lineare è un modello che non ha iperparametri, e dunque non c'è necessità di ottimizzarli. Tuttavia, è ugualmente utile fare una Cross Validation per individuare l'eventuale presenza di overfitting nel modello (pur essendo improbabile per un modello così elementare).

Un modello di questo tipo, seppur molto semplice, è molto d'aiuto per ottenere risultati interpretabili. Infatti, i coefficienti che il modello impara per ogni feature, danno una chiara spiegazione dell'importanza che ha avuto ciascuna feature nel contribuire al risultato finale della predizione. Insomma: un coefficiente β_i , ci dice che se la feature x_i viene incrementata di 1 unità, allora l'output $f(x)$ viene incrementato di β_i unità. Stabilire inoltre quale sia la classifica delle features più importanti è immediato: è sufficiente ordinarle in ordine decrescente sulla base al valore assoluto del loro coefficiente relativo.

Tuttavia, affinché i coefficienti siano utili a determinare l'importanza della features, è necessario standardizzare i dati, soprattutto dato che la prima feature è di un ordine di grandezza differente da tutte le altre.

Decision Tree

L'albero di decisione è anch'esso un modello utilizzato in questo caso in quanto interpretabile, e tuttavia ben più potente della regressione lineare.

Esso è un modello provvisto di iperparametri, ed è dunque necessario fare il loro tuning usando Cross Validation. In particolare, scelgo l'approccio di Grid Search fornito da scikitlearn, il quale al suo interno esegue una K-Folds-Cross-Validation. La Grid Search, pur andando a fare una ricerca esaustiva degli iperparametri (ed essendo dunque un metodo più pesante computazionalmente), è un approccio che ho ritenuto idoneo, poiché il numero di iperparametri di un albero di decisione da regolare non è elevato.

Gli iperparametri di cui ho fatto tuning quelli più rilevanti in un albero di decisione, ovvero:

- `max_depth` : profondità massima dell'albero.
- `min_samples_leaf` : numero **minimo** di campioni (osservazioni) che devono essere presenti in un nodo foglia.
- `ccp_alpha` : controlla quanto intensamente l'albero viene "potato" dopo essere stato costruito completamente.

Con valore 0.0 non si pota niente, mentre con valori > 0.0 , come ad esempio 0.001, o 0.01 si potano rami e l'albero di semplifica.

La scelta dei valori degli iperparametri testati è stata fatta tenendo conto che l'obiettivo è quello di ottenere un albero interpretabile, cioè non complesso ma utilizzabile per dare una spiegazione del ragionamento del modello per predire l'output. Ad esempio, la profondità massima provata è stata 5, in quando un albero più profondo di così risulta difficilmente leggibile.

Explainable Boosting Machines

L'Explainable Boosting Machines è anch'esso un modello interpretabile, e provvisto di iperparametri. È dunque necessario fare il tuning di quest'ultimi usando Cross Validation. In particolare, scelgo l'approccio di Randomized Search, che non va a fare una ricerca esaustiva nello spazio degli iperparametri, ma randomica. Questo metodo è più congeniale allo scenario delle EBM, in cui gli iperparametri da valutare sono più numerosi, e in cui il costo di training è ben più oneroso di tutti gli altri modelli interpretabili analizzati in questo progetto.

Gli EBM hanno molti iperparametri da poter variare. Per capire quali valesse la pena testare usando cross-validation, è stato molto utile l'utilizzo della documentazione presente:

Fonte: <https://interpret.ml/docs/hyperparameters.html>

Innanzitutto, si fa una distinzione importante tra iperparametri di cui non conviene fare tuning, e quelli di cui conviene farlo.

Ci sono infatti iperparametri di cui sono ben noti gli effetti sulle prestazioni del modello. Per questi, il tuning è sconsigliato e il loro valore va scelto soltanto basandosi sulle risorse e il tempo a disposizione per l'addestramento del modello. In particolare, per questi ho deciso di lasciare i valori di default, essendo quelli che bilanciano al meglio prestazioni e costo computazionale.

Per gli iperparametri di cui fare tuning invece, ho considerato l'ordine di importanza assegnato dalla documentazione. Quelli selezionati sono i seguenti:

- `max_leaves` : numero massimo di foglie negli alberi usati per ogni feature.

Le alternative possibili sono 2 e 3, così come dice la documentazione. Infatti, valori più alti comprometterebbero la interpretabilità del modello.

- `smoothing_rounds` : numero di iterazioni iniziali "conservative".

Con un valore basso si hanno curve irregolari, con rischio overfitting. Con un valore troppo alto si hanno invece curve troppo lisce, con rischio di underfitting.

La documentazione consiglia 500 dei default per problemi di regressione, ma considerando che valori più alti sembrano aumentare le prestazioni. Ho deciso quindi di estendere lo spazio di ricerca in questo modo: [350, 500, 750, 1000].

- `learning_rate` : velocità di apprendimento del modello.

La documentazione suggerisce un valore di default pari a 0.04 per regressione, ma anche la relazione tra `learning_rate` e prestazioni finali non è del tutto chiara. Per cui, ho deciso di estendere il raggio di ricerca ad un'ampia gamma di valori presentati dalla documentazione, ovvero [0.02, 0.03, 0.04, 0.05, 0.1].

- `interactions` : numero di interazioni tra features.

Questo è l'iperparametro più delicato e determinante ai fini dell'interpretabilità del modello risultante.

Aumentare il numero di interazioni, significa aumentare la flessibilità del modello, ma anche ridurre l'interpretabilità.

Per fissare le idee, è sufficiente pensare che un numero di interazioni pari a 0 indica che il modello è puramente additivo, cioè che ogni funzione imparata dipende da una sola feature.

Un esempio in formule è il seguente:

$$EBM = f_1(x_1) + f_2(x_2) + f_3(x_3)$$

Viceversa, aumentando le interazioni, posso imparare funzioni che dipendono anche da più features.

Un esempio in formule è il seguente:

$$EBM = f_1(x_1) + f_2(x_2) + f_{12}(x_1, x_2)$$

L'esecuzione dell'addestramento del modello con randomized-search su questi 4 iperparametri ha richiesto all'incirca 10 ore. Ho quindi deciso di non proseguire oltre nel coinvolgere altri iperparametri, in quanto i rimanenti sono possibilmente ancora più impattanti sulle tempistiche di addestramento, e con ritorni sicuramente minori in termini di performances (così come espresso dalla documentazione).

Seconda fase - implementazione e confronto con Heteroskedastic Likelihood and Multi-Latent GP

Richiamo sulla nozione di eteroschedasticità

Tra le assunzioni che si fanno sui dati per poter applicare correttamente i modelli di Linear Regression, ce n'è una che riguarda la varianza degli errori commessi, vale a dire:

| La varianza degli errori è costante per tutti i valori della feature indipendente x .

Se poi ci sono più features, questa proprietà, che prende il nome di **omoschedasticità**, deve valere per ciascuna feature.

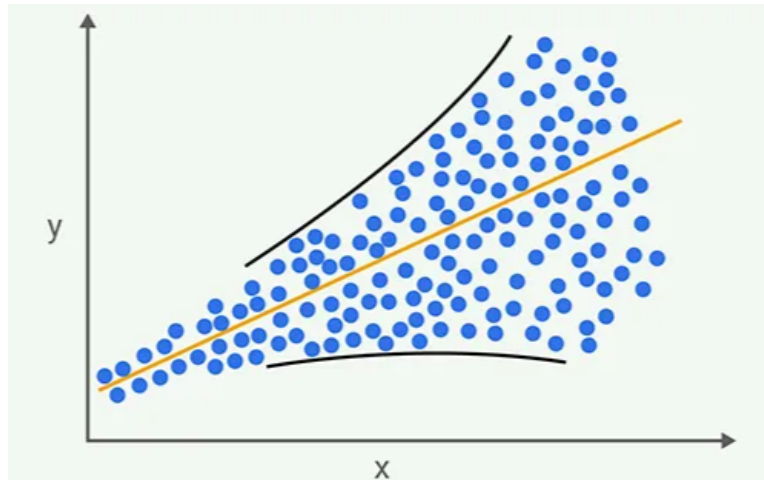
La presenza di questa proprietà non è ovviamente garantita, e va verificata con dei test appositi. Nel caso non valga, si dice che siamo in presenza di **eteroschedasticità**, la situazione opposta.

Possiamo esprimere graficamente questo concetto con un semplice esempio. Consideriamo un grafico in cui sull'asse delle ascisse è presente l'altezza (la feature) e sull'asse delle ordinate il peso da predire (il target).

È verosimile pensare che individui con altezza intorno ai 50 cm (dunque neonati) abbiano per lo più pesi simili. Questo si traduce in una varianza di errore bassa per quei valori della feature.

Viceversa, è ragionevole anche notare che per individui con altezza intorno a 175 cm (dunque adulti) ci siano molti valori di peso possibili corrispondenti. Questo si traduce in una varianza di errore alta per quei valori della feature.

Dunque, graficamente, si ottiene un grafico con una forma a ventaglio, in cui i punti degli esempi sono più vicini per certi valori di feature, e sono più rarefatti per altri valori di feature.



Fonte: <https://ppcexpo.com/blog/what-is-heteroskedasticity>

Da un punto di vista matematico, in una situazione di eteroschedasticità, non è sufficiente un valore costante per la varianza, solitamente chiamato σ^2 . Di fatti, anche la varianza, varia a seconda del valore della feature considerata.

Test per individuare eteroschedasticità

Andrò ad eseguire due tipologie di test, uno grafico ed uno statistico.

Chiaramente le eseguirò soltanto sul train set di ciascuno dei 4 dataset, non toccando i test set che infatti non vanno mai utilizzati se non per la valutazione di generalizzazione dei modelli.

Test del residual plot - Test grafico

Il test grafico consiste nell'andare a visualizzare un residual plot, per poi andare ad analizzare la forma del plot risultante.

A questo punto si distinguono due casi:

- Se la forma del plot è simile ad una nuvola randomica di rumore senza pattern individuabili al suo interno, allora è probabile che non ci sia eteroschedasticità.
- Se la forma del plot è simile ad una forma a ventaglio, allora è presente eteroschedasticità.

Test di Breusch-Pagan - Test statistico

Per quanto riguarda il test statistico, quello che applicherò è il cosiddetto test di Breusch-Pagan.

Una implementazione robusta del suddetto test è presente nella libreria *statsmodels*, che andrò quindi ad utilizzare. Per la precisione, importerò il pacchetto `statsmodels.stats.diagnostic`.

Il risultato del test ha la seguente interpretazione:

- Se $p - value \leq 0.05$:
allora i dati presentano eteroschedasticità.
- Se $p - value \geq 0.05$:
allora i dati presentano omoschedasticità.

Risultati dei due test

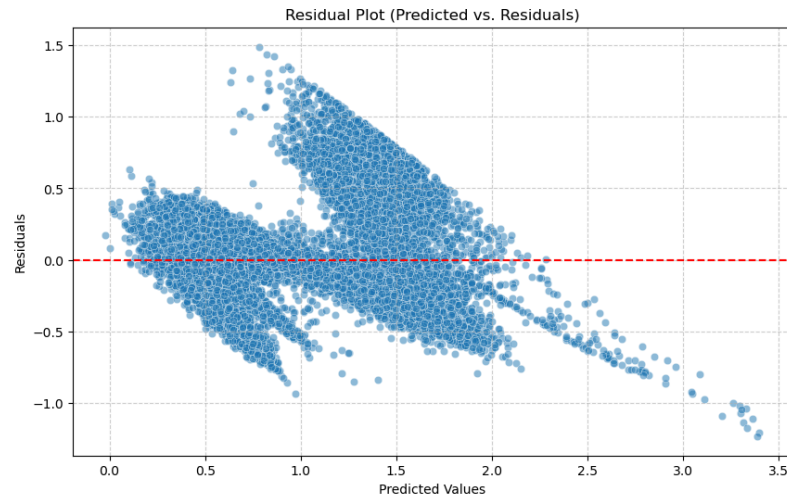
Tutti i plot visualizzati suggeriscono la presenza di eteroschedasticità nei dati.

I residui infatti sono distribuiti non come punti rumorosi randomici, ma hanno invece una forma ben precisa, presentando pattern e cluster facilmente individuabili.

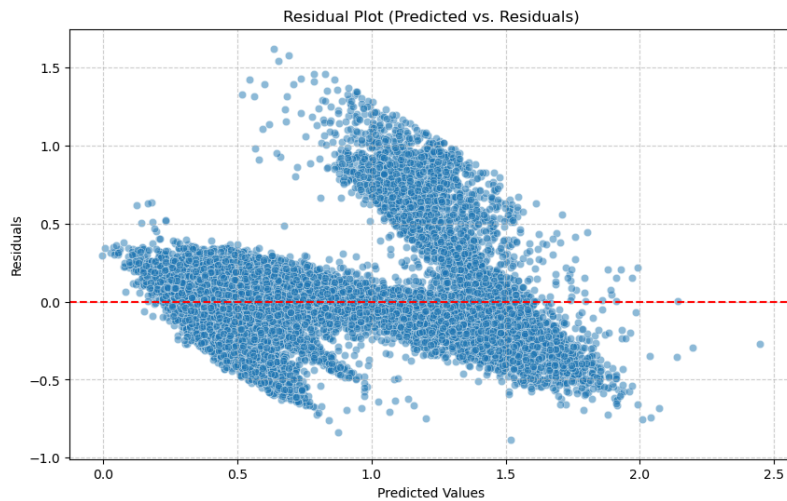
In altre parole, la dispersione dei residui cambia al variare dei valori predetti, con una forma che in alcune zone è più stretta, ed in altre è più ampia, cosa che indica come la varianza negli errori non sia costante, ma vari.

Di seguito, i residual plots ottenuti:

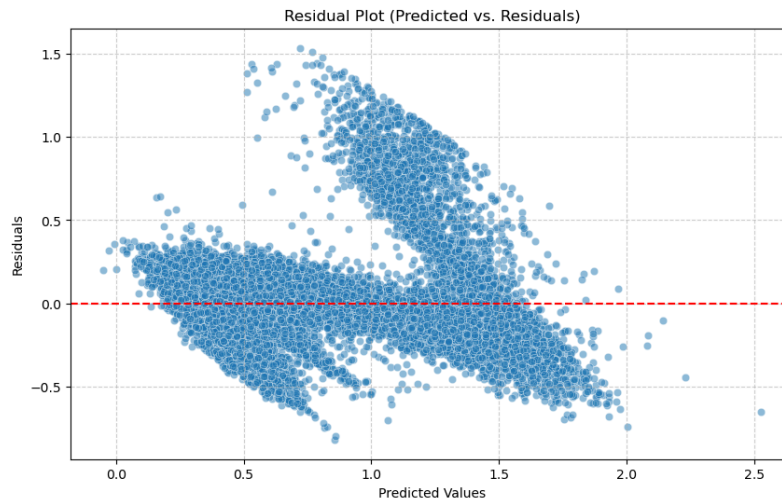
Dataset `df_train_noCS`



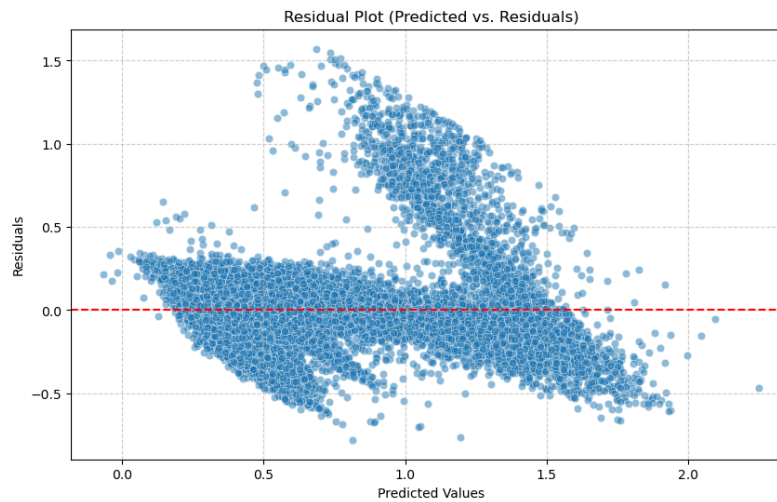
Dataset `df_train_weakCS`



Dataset `df_train_mildCS`



Dataset `df_train_strongCS`



L'evidenza del test grafica, è poi confermata dal test statistico di Breusch-Pagan. In ciascun test effettuato, uno per ognuno dei 4 train set, il test ha riportato un forte grado di eteroschedasticità nei dati. Difatti, il risultato è stato un p-value di 0.0000 per tutti.

	df_train_noCS	df_train_weakCS	df_train_mildCS	df_train_strongCS
p-value	0.0000	0.0000	0.0000	0.0000

Il risultato dei due test dunque è coerente, e conferma la presenza di eteroschedasticità nei dati. Questo rende sensata la prova dell'ultimo metodo da valutare, cioè quello di *Heteroskedastic Likelihood and Multi-Latend GP*, appositamente costruito per lavorare su dati eteroschedastici.

Implementazione del metodo di Heteroskedastic Likelihood and Multi-Latend GP

Prima di addentrarsi nell'implementazione del modello per dati eteroschedastici, conviene rivedere brevemente che cosa sia un Gaussian Process nella sua formulazione di base, cioè per dati omoschedastici, e poi la sua estensione per dati eteroschedastici.

Gaussian Process standard

Un Gaussian Process è un modello probabilistico che invece di predire un singolo valore, predice un'intera distribuzione di probabilità sull'output.

In pratica, per ogni punto predetto, non fornisce solo il valore della predizione (ad es. "la predizione è 2.1") ma fornisce anche le informazioni di confidenza attorno a quel punto (ad es. "la predizione segue una distribuzione Gaussiana con media 5.2 e deviazione standard 0.3").

Questo lo rende naturalmente capace di quantificare l'**incertezza** delle predizioni effettuate.

Un processo gaussiano standard esprime le predizioni così:

$$y \sim \mathcal{N}(f(\mathbf{x}), \sigma^2)$$

dove $f(\mathbf{x})$ è la funzione che impara la media delle predizioni (come un qualsiasi regressore), e σ^2 è la varianza degli errori commessi che si assume costante per ogni punto $\mathbf{x}$.

Il limite di questo modello è proprio l'assunzione che fa sulla varianza. Non è vero che infatti essa sia sempre costante, ma anzi, in casi come quello della predizione di photo-z questo non vale, poiché abbiamo eteroschedasticità nei dati.

Gaussian Process per dati eteroschedastici

La soluzione, per tenere conto della varianza non costante, si basa sull'estensione del modello. Dunque invece di imparare soltanto una funzione della media dei valori predetti ($f_1(\mathbf{x})$), si impara anche una funzione che rappresenta quanto è incerta la predizione in quel punto ($f_2(\mathbf{x})$).

Un processo gaussiano per dati eteroschedastici esprime le predizioni così:

$$y \sim \mathcal{N}(f_1(\mathbf{x}), \exp(f_2(\mathbf{x}))^2)$$

dove $f_1(\mathbf{x})$ apprende la media delle predizioni in funzione dell'input $\mathbf{x}$, e $f_2(\mathbf{x})$ apprende la varianza delle previsioni in funzione dell'input $\mathbf{x}$.

Si noti poi come l'esponenziale venga utilizzato per fare in modo di non avere mai una deviazione standard (cosa che in pratica è $f_2(\mathbf{x})$) negativa.

Il risultato che si ottiene è che il modello può predire **incertezze diverse in zone diverse** dello spazio degli input, ed è proprio quello che serve con dati eteroschedastici.

Utilizzo di SVGP per rendere il problema trattabile

I gaussian processes classici hanno un problema: scalano molto male con i dati. Difatti, essi hanno complessità computazionale pari a $O(N^3)$ sia in training che in inferenza, dove N è il numero di campioni.

Questo è un problema molto rilevante, perché già solo con dataset di qualche migliaio di punti, il loro utilizzo diventa computazionalmente proibitivo. Dato che i dataset da utilizzare nel caso della predizione di photo-z sono di diverse migliaia di punti, serve una soluzione.

La soluzione adottata nell'articolo di riferimento è **SVGP (Sparse Variational Gaussian Process)**. L'idea è semplice: invece di usare tutti gli N punti del dataset, il modello usa un sottoinsieme di M **inducing points** ($M \ll N$) che approssimano il comportamento del Processo Gaussiano completo. Nella documentazione vengono ad esempio usati $M = 20$ inducing points.

Tuning degli iperparametri

Gli iperparametri più significativi, sia in termini di performances del modello, che di tempo necessario al training, sono i seguenti:

- `M`: numero di inducing points
- `epochs`: numero di epoche di training

Per un tuning corretto, ho eseguito uno split del dataset di training intero in due ulteriori dataset, cioè quello di training "effettivo", e quello di validation.

A questo punto, le considerazioni sulla determinazione dei migliori iperparametri le ho fatte utilizzando questi due dataset, utilizzando il primo per l'addestramento, e il secondo per il calcolo delle prestazioni. Questo ha permesso di non toccare il test set, il quale va usato solo per la valutazione finale del modello.

Il tuning dei due iperparametri potrebbe essere fatto separatamente, trovando prima il miglior valore di `M` per un valore di epoche costante, e poi trovando il valore migliore delle epoche per l'`M` scelto. In realtà questo non porterebbe ad un tuning rigoroso.

Difatti, i due iperparametri non sono indipendenti tra loro, ma dipendono fortemente uno dall'altro. È cosa nota in letteratura, che valori inferiori di `M` necessitano di un numero di epoche superiore prima di arrivare a convergenza (e poi di conseguenza ad overfitting). Valore superiori di `M` invece, necessitano di un numero di epoche inferiore prima di arrivare allo stesso punto.

Dunque, per fare una valutazione che tenga conto di questo, la strategia è stata la seguente.

Ho selezionato dei valori di `M` come spazio tra cui eseguire la ricerca.

La scelta è ricaduta su: `M = [20, 100, 200, 1000]`

Non sono andato oltre, perché in letteratura è emerso che valori di inducing points oltre al 5% della dimensione del dataset di training, non portano vantaggi di prestazioni considerevoli, in confronto all'aumento del tempo di esecuzione che esplode.

Per ciascun valore di `M` ho quindi creato un modello che è stato poi addestrato con early stopping, valutandolo in base alle prestazioni del modello sul validation set.

L'early stopping è stato utilizzato per evitare che il modello arrivasse ad overfitting, ed è stato settato con una patience di 100 epoche, ed una loss (calcolata da `loss_fn()` del modello, che è la **Negative ELBO (Evidence Lower Bound)**).

È rilevante fare una precisazione ulteriore sull'early stopping. Esso avviene nel momento in cui scocca la centesima epoca consecutiva in cui la validation loss non è migliorata. Questo implica che a quel punto, il modello attuale non sia il migliore possibile, in quanto addestrato troppo. La soluzione consiste allora nel ripristinare il modello (di cui si è salvato un checkpoint) migliore trovato fino a quel momento, una volta verificatosi l'early stopping.

A questo punto, si ottengono i risultati per ciascuna configurazione del modello. Si va a selezionare poi il modello migliore sulla base delle prestazioni sempre sul validation set.

Selezionato il modello migliore, non c'è bisogno di riaddestrarlo, ma lo si va direttamente a valutare sul test set.

Focus su tecniche di explainability utilizzate

Come già detto, parte rilevante di questo progetto è stata l'interpretabilità dei risultati dati dai modelli.

Sono stati utilizzati vari modelli interpretabili con conseguenti metodologie di spiegazione. Tutte cercano di assegnare un'importanza alle features, determinando in che modo il loro valore impatti sul risultato.

Explainability con regressione lineare

Di seguito vengono riportati i coefficienti dei modelli addestrati di regressione lineare. È presente una tabella per ciascun dataset, in cui all'interno le features sono ordinate dalla più importante alla meno importante basandosi sul modulo del coefficiente di regressione relativo. Un modulo più grande implica una feature più importante, e viceversa.

Dataset `train_noCS`

	Feature	Importance	Importance_abs
5	zy	0.251859	0.251859
4	iz	0.202397	0.202397
2	gr	-0.192447	0.192447
0	r	0.113586	0.113586
3	ri	-0.071009	0.071009
1	ug	0.026390	0.026390

Dataset `train_weakCS`

	Feature	Importance	Importance_abs
5	zy	0.211286	0.211286
2	gr	-0.178847	0.178847
4	iz	0.168190	0.168190
0	r	0.095952	0.095952
1	ug	-0.015123	0.015123
3	ri	0.002007	0.002007

Dataset `train_mildCS`

	Feature	Importance	Importance_abs
5	zy	0.204914	0.204914
2	gr	-0.171726	0.171726
4	iz	0.162354	0.162354
0	r	0.091793	0.091793
1	ug	-0.025759	0.025759
3	ri	0.012986	0.012986

Dataset `train_strongCS`

	Feature	Importance	Importance_abs
5	zy	0.184119	0.184119
2	gr	-0.161140	0.161140
4	iz	0.155470	0.155470
0	r	0.090481	0.090481
1	ug	-0.031097	0.031097
3	ri	0.027694	0.027694

Si può notare come all'aumentare della CS non ci siano sostanziali variazioni nell'importanza delle features. La feature più importante rimane sempre `zy`, e le prime tre posizioni rimangono pressoché le stesse, tranne che per un'inversione tra `iz` e `gr` nel caso di assenza di CS.

Explainability utilizzando albero di decisione

Per la visualizzazione degli alberi stampati graficamente, si rimanda il lettore a visionare l'output del notebook, data la difficoltà dell'inserimento degli stessi nella relazione.

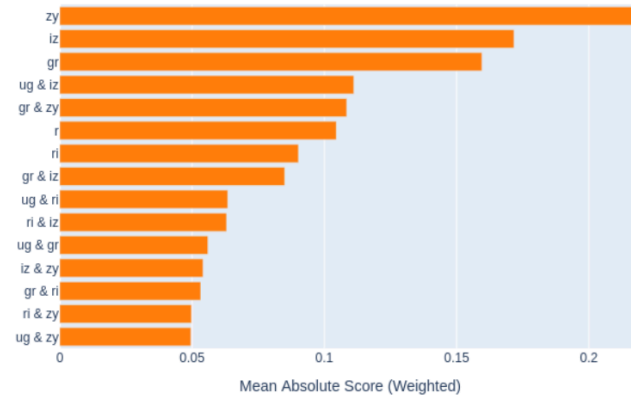
Explainability utilizzando Explainable Boosting Machines

Di seguito, per ciascun dataset viene riportato il grafico della funzione imparata per ciascuna feature, oltre ad un grafico con il riassunto delle feature più importanti.

Dataset `train_noCS`

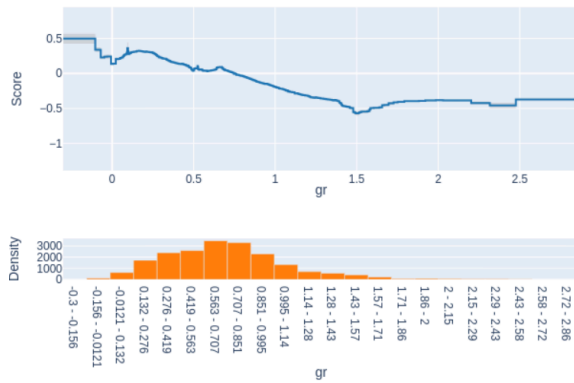
Riassunti dell'importanza delle features e delle coppie di features più importanti:

Global Term/Feature Importances

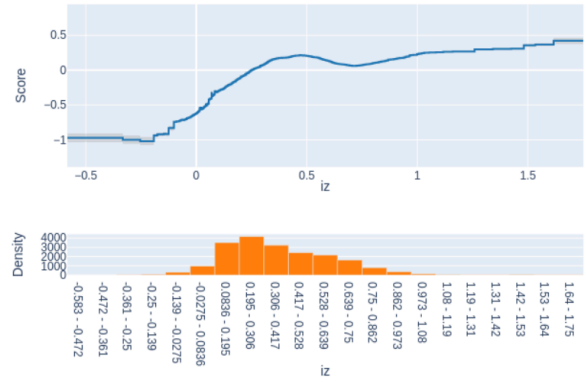


Funzioni imparate per ciascuna feature:

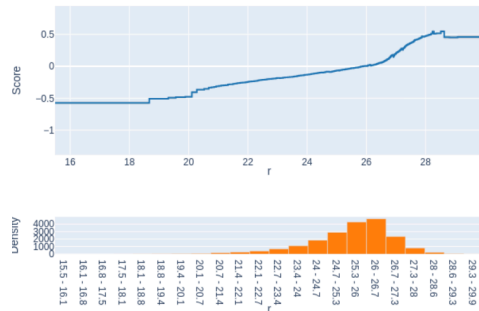
Term: gr (continuous)



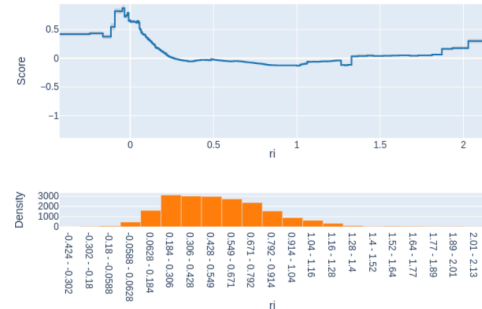
Term: iz (continuous)



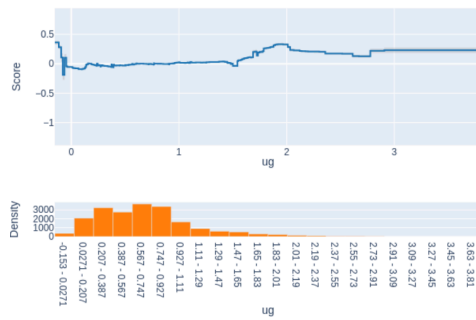
Term: r (continuous)



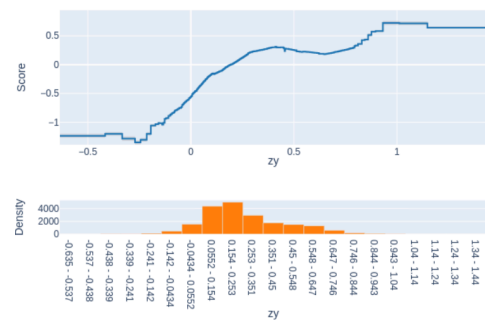
Term: ri (continuous)



Term: ug (continuous)



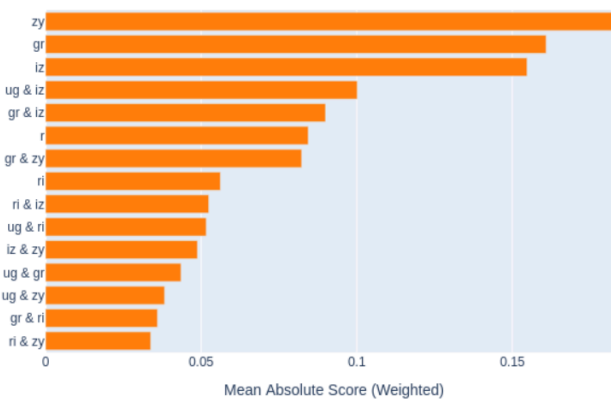
Term: zy (continuous)



Dataset `train_weakCS`

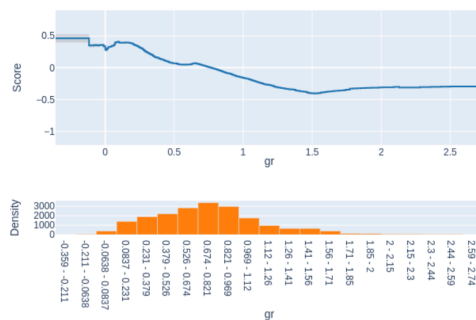
Riassunto dell'importanza delle features e delle coppie di features più importanti:

Global Term/Feature Importances

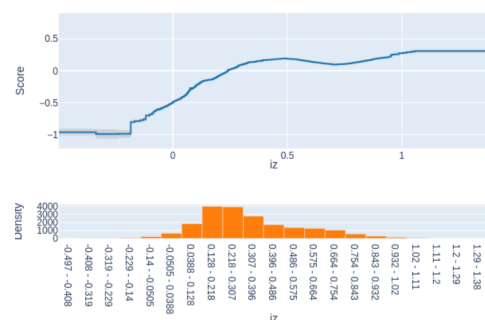


Funzioni imparate per ciascuna feature:

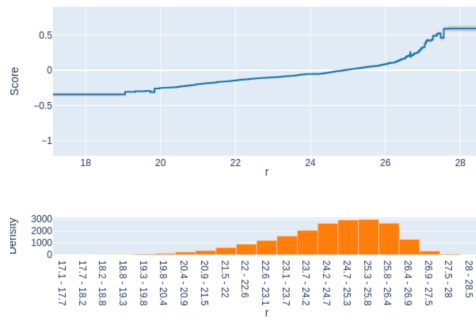
Term: gr (continuous)



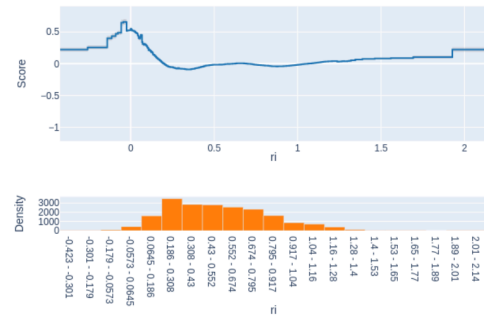
Term: iz (continuous)



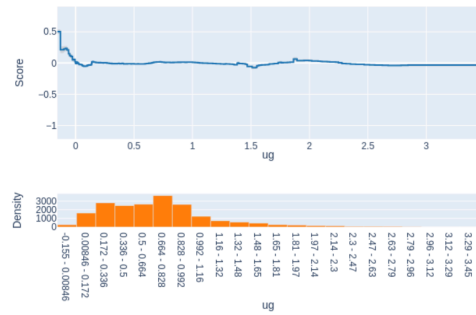
Term: r (continuous)



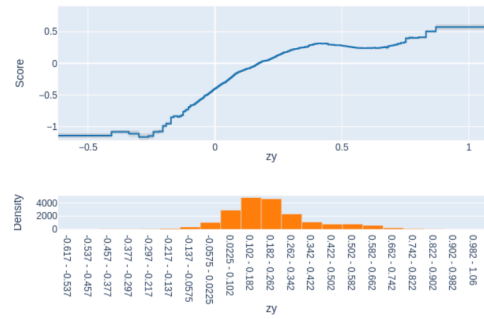
Term: ri (continuous)



Term: ug (continuous)



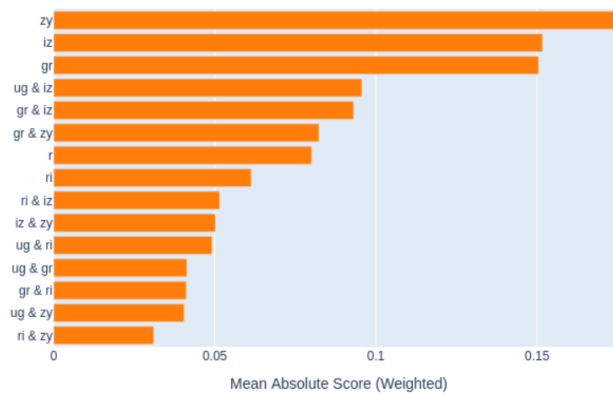
Term: zy (continuous)



Dataset `train_mildCS`

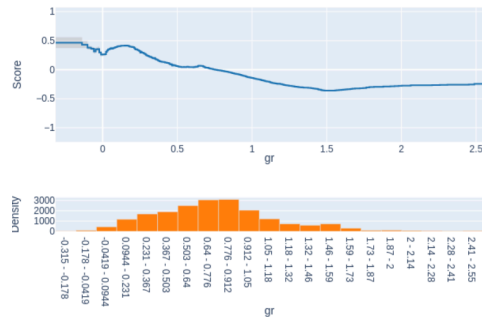
Riassunto dell'importanza delle features e delle coppie di features più importanti:

Global Term/Feature Importances

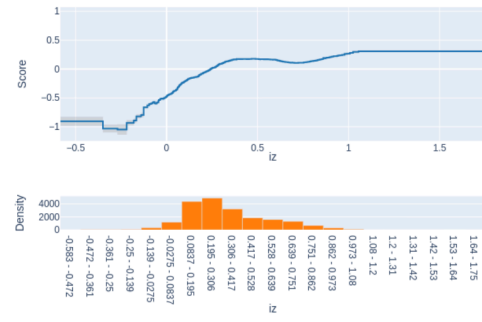


Funzioni imparate per ciascuna feature:

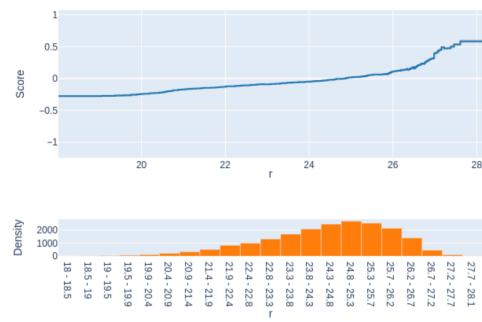
Term: gr (continuous)



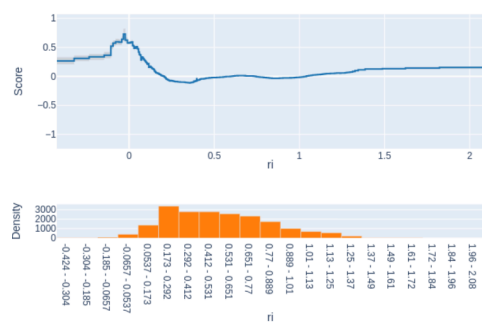
Term: iz (continuous)



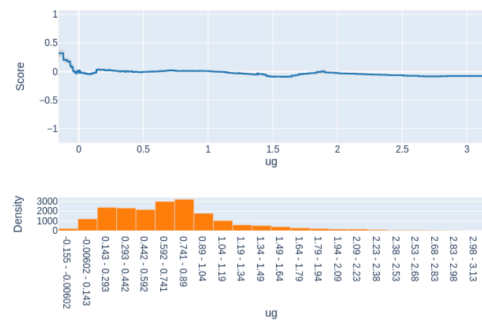
Term: r (continuous)



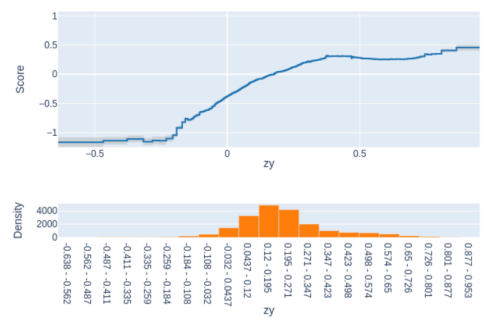
Term: ri (continuous)



Term: ug (continuous)



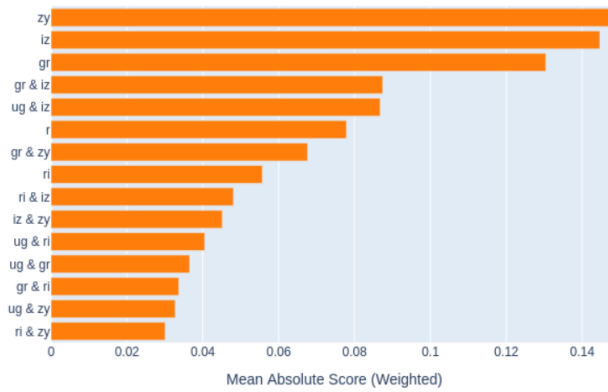
Term: zy (continuous)



Dataset `train_strongCS`

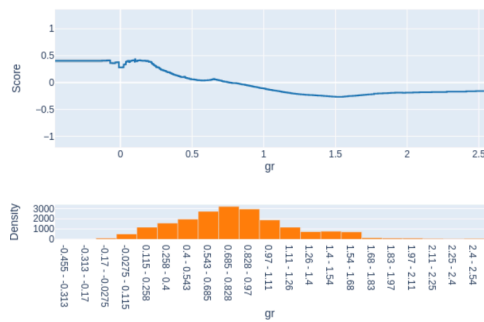
Riassunto dell'importanza delle features e delle coppie di features più importanti:

Global Term/Feature Importances

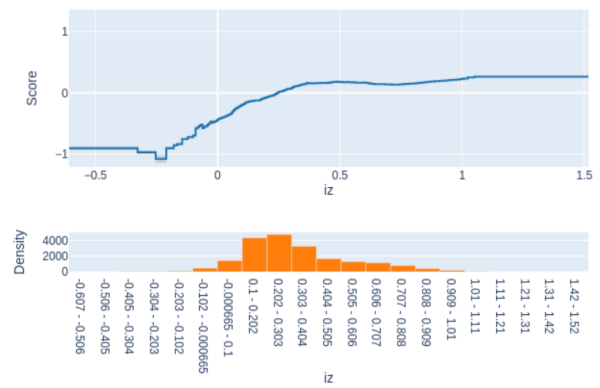


Funzioni imparate per ciascuna feature:

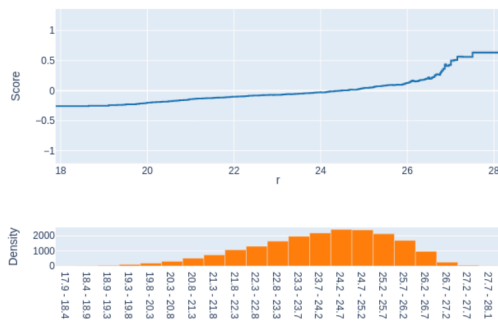
Term: gr (continuous)



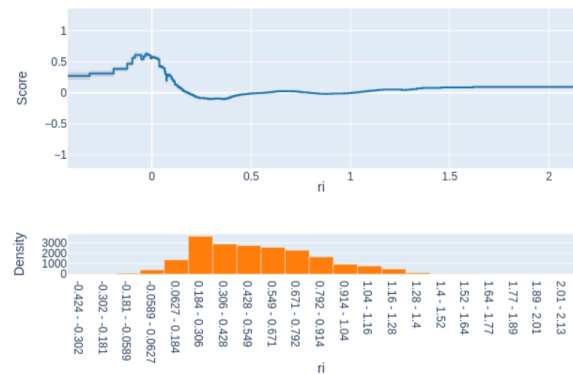
Term: iz (continuous)

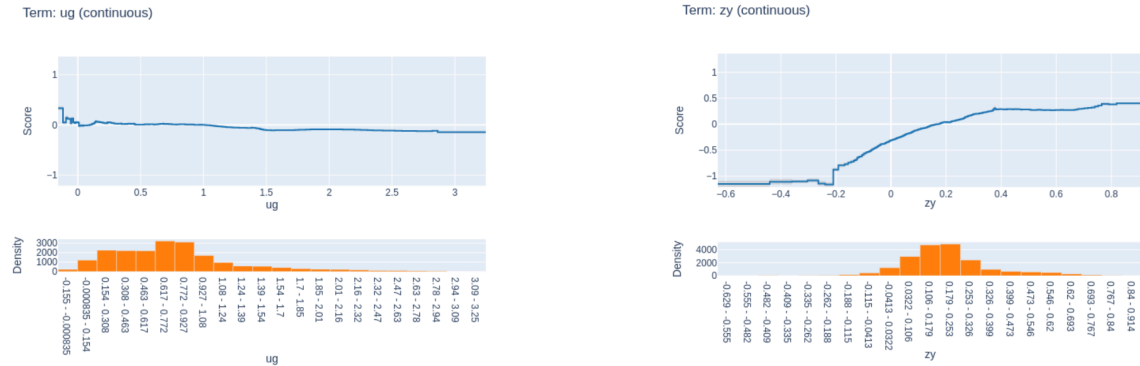


Term: r (continuous)



Term: ri (continuous)





La prima tipologia di grafico descrive in maniera riassuntiva quali siano le features più importanti ai fini delle predizioni. Anche in questo caso, si conferma il risultato visibile negli altri metodi di spiegabilità, notando come feature più importante **zy**, e al secondo e terzo posto le features **iz** e **gr**.

Rispetto però ai risultati di SHAP che verranno presentati subito dopo questi, si nota come la differenza tra l'importanza della prima feature e della seconda feature sia qui meno marcata. Stessa cosa poi vale per la differenza tra seconda e terza feature, anche loro più allineate.

Si noti che queste osservazioni valgono per ciascun dataset esaminato.

La seconda tipologia di grafico descrive invece per ogni feature, nella porzione superiore la funzione imparata sulla feature, che descrive il contributo della feature sulla predizione del modello; viceversa, nella porzione inferiore, descrive la quantità di esempi presenti nel dataset relativi ai valori della feature.

Globalmente, l'andamento di questi grafici per ciascuna feature e per ciascun dataset, è concorde con ciò che riportano i beeswarm plots del metodo SHAP, visibili nella sezione successiva.

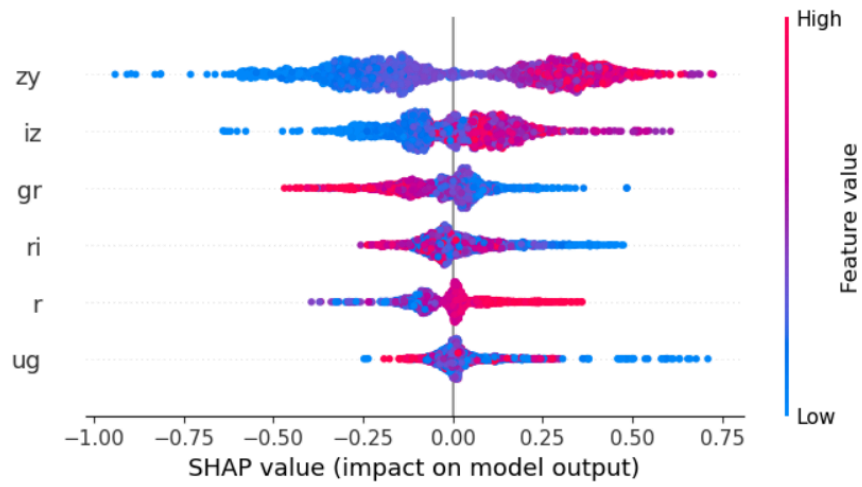
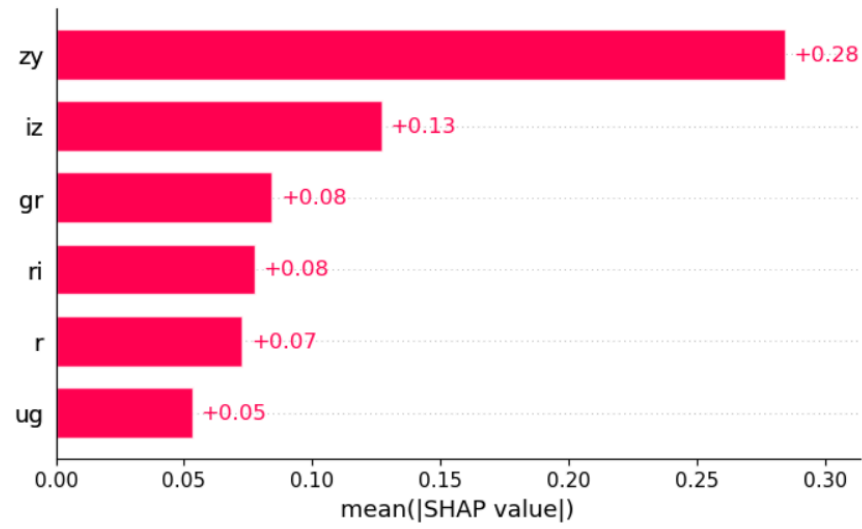
Explainability utilizzando SHAP

Ho implementato un'ulteriore metodologia di explainability sui dati da confrontare con quelle dei modelli precedenti. È stato scelto SHAP, in quanto una delle metodologie di explainability più utilizzate, essendo anche model agnostic.

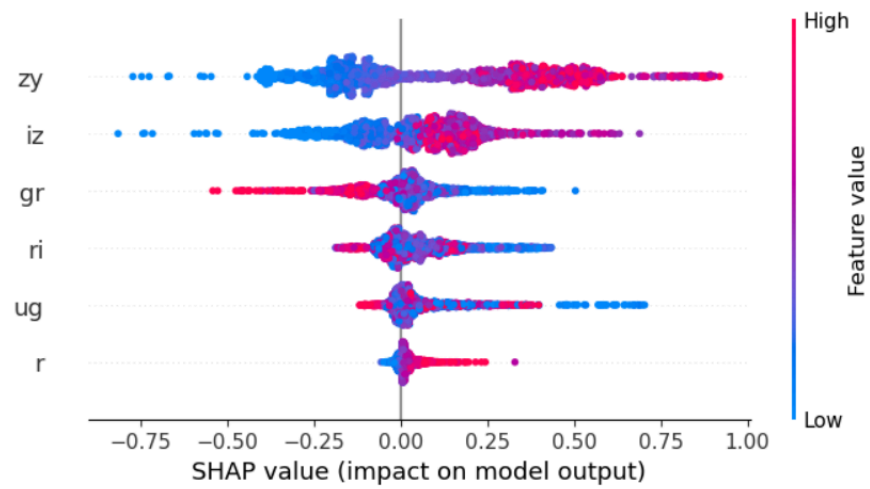
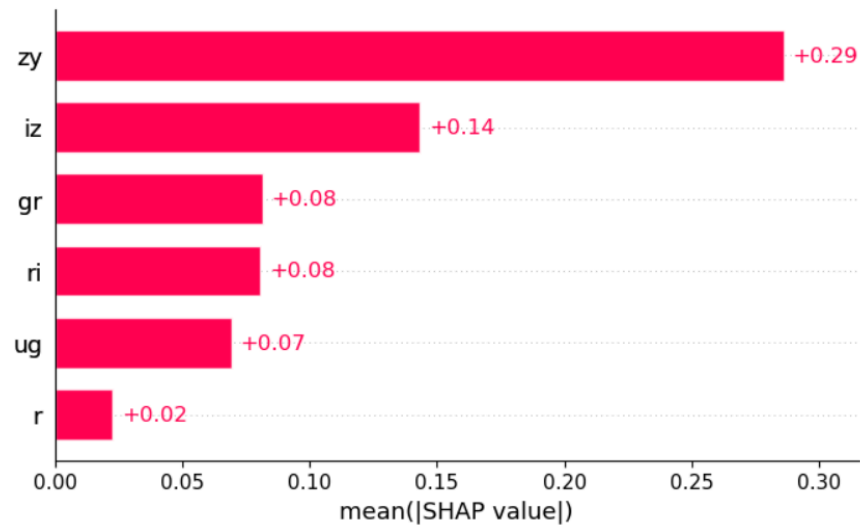
Per quanto riguarda il modello di random forest, viene utilizzato quello con i parametri di default dalla libreria scikit-learn, e questo perché non è un modello di cui ci interessa confrontare le performances con gli altri. Ci interessa infatti come appoggio per una ulteriore baseline di spiegabilità, garantita dal metodo SHAP.

Di seguito per ciascun dataset di test vengono riportati due grafici. Il primo a barre riporta l'importanza delle features in base al modulo dello shapley value corrispondente, mentre il secondo è un classico beeswarm.

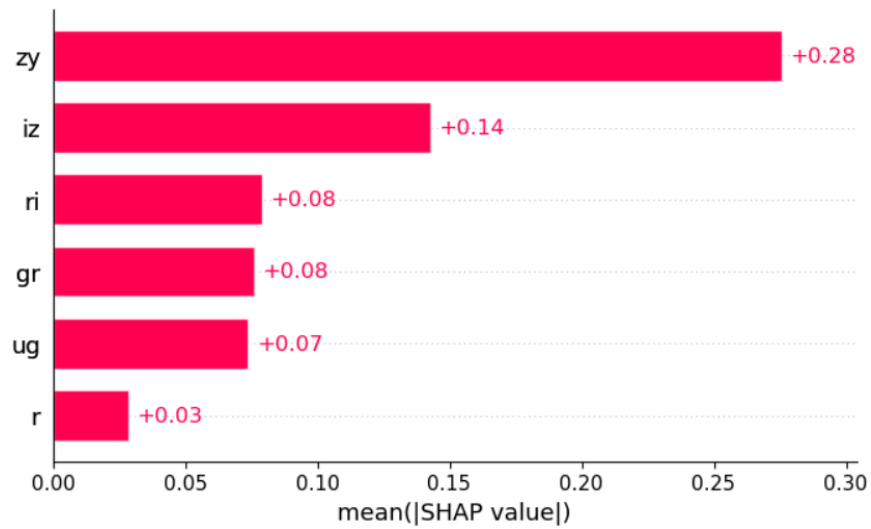
Dataset **test_noCS**

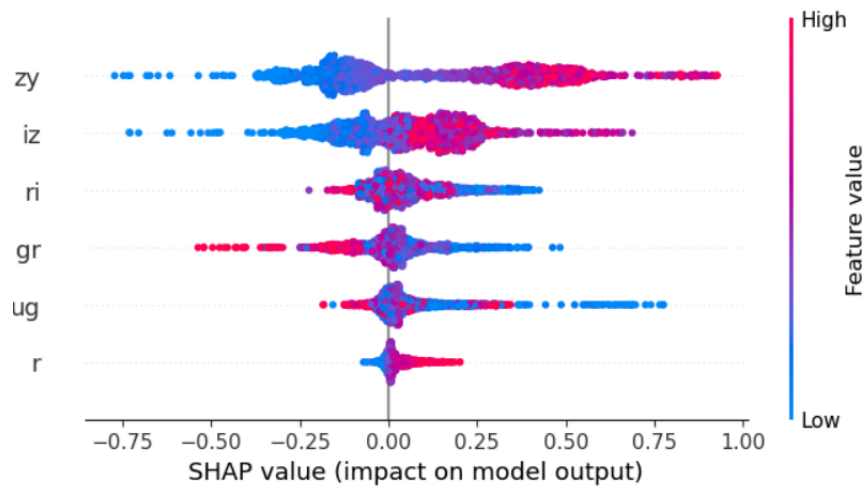


Dataset `test_weakCS`

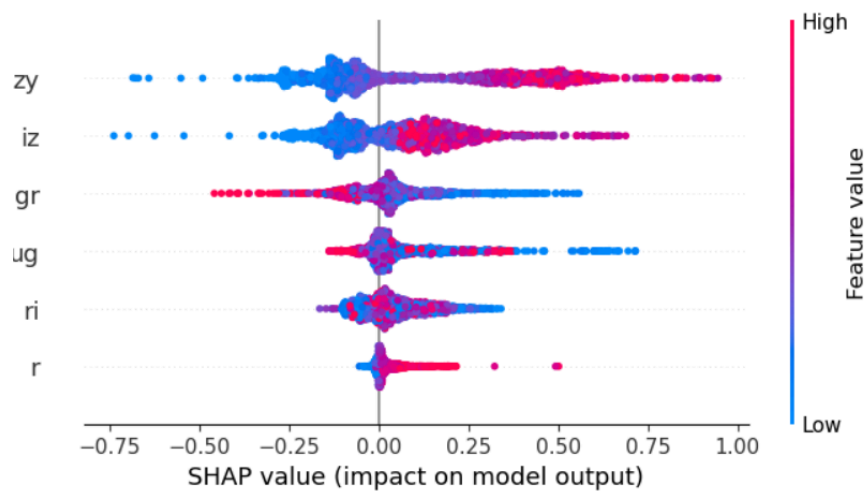
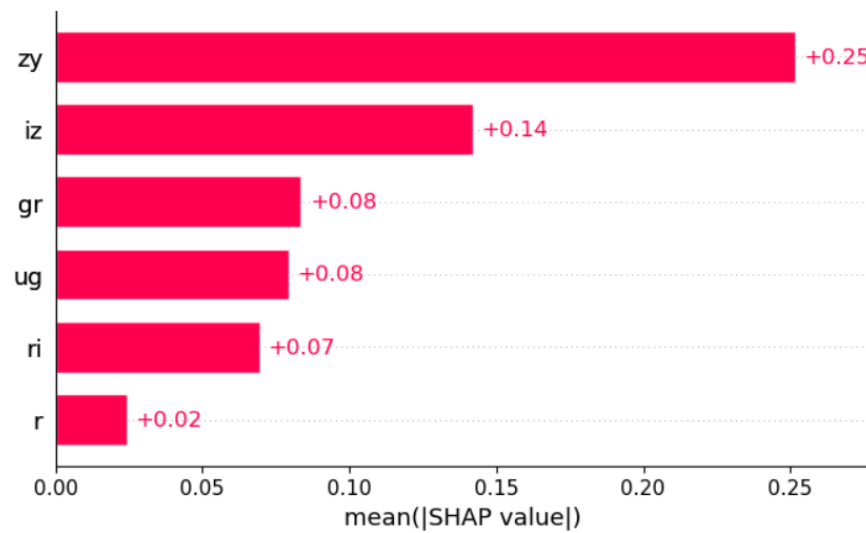


Dataset `test_mildCS`





Dataset `test_strongCS`



Dai grafici a barre si può notare come la feature `zy` sia quella che in valore assoluto (dunque sia in positivo che in negativo) impatta di più in media sul dataset il risultato finale. Questo vale inoltre per tutti e quattro i dataset, a prescindere dal grado di covariante shift.

Ben staccata, troviamo in seconda posizione la feature `iz`, mentre le rimanenti hanno circa la stessa importanza globale, con l'unica eccezione della feature `r` che all'aumentare di covariate shift, diventa in assoluto meno importante. Ancora una volta questo, lo si apprezza per tutti i dataset.

I beeswarm plots entrano poi più nel dettaglio, e rendono esplicita la relazione tra valori delle features e valore predetto dal modello. Ricordo come ciascun punto grafico posizionato in linea con ciascuna feature, corrisponda al valore dello shapley value di quella feature per un particolare esempio del dataset. Sezioni più larghe corrispondono a punti con stesso shapley value per la feature considerata. I beeswarm non presentano molte differenze tra i vari dataset, con un'unica eccezione per la sezione relativa agli shapley values della feature `r`. Si può notare infatti come nel passaggio da dataset `noCS` (quindi con assenza di CS) a dataset `weakCS` (quindi con presenza di CS), la forma del plot vari di molto.

Se nel primo essa presenta una ampia coda sinistra con valori per lo più bassi di `r`, indice di un impatto considerevole su una buona quantità di esempi per quanto riguarda lo shapley value risultante, nel secondo essa presenta una coda sinistra con valori alti di `r` molto limitata, indice di valori bassi di `r` che hanno impatto bassissimo o nullo nella determinazione globale della predizione da parte di quella feature. Questa ultima tendenza inoltre, si mantiene stabile anche negli altri dataset, conferma ulteriore che l'aumento di CS non ha un impatto pesante quanto il passaggio da assenza di CS a presenza di CS.

Risultati e confronto tra modelli

Le performances dei modelli sono riassunte nelle tabelle sottostanti.

Ciascuna tabella riassuntiva contiene i risultati ottenuti da ciascun modello, per ogni dataset analizzato (e dunque per ciascun grado di CS).

Le performance dei modelli sono state valutate con due metriche che sono il *mean squared error*, ed il *root mean squared error*.

Tabella dei risultati per mean squared error (MSE):

	Modello	test_noCS	test_weakCS	test_mildCS	test_strongCS
0	Baseline	0.329365	0.438801	0.451955	0.464708
1	Regressione Lineare	0.118941	0.150276	0.153320	0.151714
2	Albero di decisione	0.055664	0.068554	0.075952	0.068399
3	Explainable Boosting Machine	0.009475	0.014358	0.015678	0.019892
4	GP for Heteroskedastic data	0.009044	0.056429	0.058714	0.051393

Tabella dei risultati per root mean squared error (RMSE):

	Modello	test_noCS	test_weakCS	test_mildCS	test_strongCS
0	Baseline	0.573903	0.662421	0.672276	0.681695
1	Regressione Lineare	0.344879	0.387655	0.391561	0.389504
2	Albero di decisione	0.235931	0.261829	0.275594	0.261532
3	Explainable Boosting Machine	0.097338	0.119824	0.125212	0.141038
4	GP for Heteroskedastic data	0.095099	0.237548	0.242309	0.226701

Si nota subito intanto, come la previsione iniziale sul fatto che i modelli peggiorino all'aumentare del grado di covarianza shift, sia confermata quasi del tutto. L'unica eccezione la si ha per il dataset `mildCS` nella regressione lineare e nell'albero di decisione. Qui infatti le prestazioni sono effettivamente inferiori di quelle ottenute nel dataset con `strongCS`.

Subito dopo, risulta evidente quanto la prestazione dei modelli peggiori maggiormente nel passaggio da `noCS` a `weakCS`, più di quanto peggiori all'aumentare ulteriore del grado di CS. In altre parole, si può notare come sia più negativo in termini di prestazioni dei modelli il passaggio da assenza di CS a presenza di CS, rispetto al passaggio da bassi livelli di CS, ad alti livelli di CS.

Veniamo adesso alle performances dettagliate dei modelli.

Il modello migliore per il dataset `noCS` si è rivelato essere il GP for Heteroskedastic data, mentre subito dietro a distanza minima troviamo l'Explainable Boosting Machine.

Sugli altri dataset, la storia è invece molto diversa. All'aumentare del grado di CS, il modello che ha performato meglio è stato sempre l'Explainable Boosting Machine.

Non solo inoltre è stato il miglior modello in tutti i dataset afflitti da CS, ma è stato quello che ha anche mantenuto le proprie performance più stabili degradando meno degli altri.

Conclusione

Il modello con prestazioni migliori in presenza di CS si è rivelato l'Explainable Boosting Machine. Subito dopo troviamo il modello GP for Heteroskedastic data, e più staccati -come prevedibile- gli altri modelli interpretabili.

Vale la pena ricordare le tempistiche di addestramento (su tutti e 4 i dataset complessivamente) per un confronto più equo.

Il tempo di addestramento ed predizione del modello di EBM è stato all'incirca 10 ore, seguito dal modello GP con all'incirca 7 ore di training. Gli altri modelli invece hanno tempistiche trascurabili.

Una nota importante sulla explainability. Ricordo che il modello di EBM è interpretabile by design, mentre il modello GP non lo è. Ad ogni modo, è capace di restituire un output più completo dei normali modelli, in quanto oltre alla predizione puntuale, restituisce anche la confidenza sottoforma di distribuzione di probabilità.

Dunque, possiamo concludere come il migliore modello tra quelli analizzati per la predizione del valore di photo-z, confrontando tempistiche di addestramento, interpretabilità dei risultati, e soprattutto prestazioni, sia l'Explainable Boosting Machine. Quest'ultimo è il modello che non solo ha ottenuto le migliori prestazioni su

tutti i datasets contententi covariate shift, ma è anche quello che ha visibilmente subito una minore degradazione in termini di prestazioni all'aumentare del grado di covariate shift.

Non è da escludere la possibilità di ulteriori marginali miglioramenti nel modello GP for heteroskedastic data, ma sono comunque limitati dal fatto che gli iperparametri di cui si è fatto tuning (n° inducing points ed epoche di addestramento) siano quelli più impattanti, oltre che dal fatto che all'aumentare ulteriore della frazione del numero di inducing points rispetto agli esempi totali di training, si raggiunga un plateau sostanziale nelle prestazioni, rispetto ad un aumento considerevole del tempo di training del modello.

In ultima istanza, riporto un confronto dei risultati ottenuti con le performance del metodo *StratLearn-z* descritto nell'articolo principale da cui sono stati prelevati i dati utilizzati.

Tra le metriche di performance riportate nell'articolo, prendo ad esame l'RMSE, essendo una delle metriche utilizzate proprio anche in questo progetto per analizzare i risultati dei modelli.

Di seguito quindi la porzione di interesse della tabella riassuntiva dei risultati presente nell'articolo *StratLearn-z*:

	df_train_noCS	df_train_weakCS	df_train_mildCS	df_train_strongCS
RMSE	0.0583	0.0697	0.0724	0.0817

E riporto anche la tabella dei risultati RMSE per agevolare il confronto:

	Modello	test_noCS	test_weakCS	test_mildCS	test_strongCS
0	Baseline	0.573903	0.662421	0.672276	0.681695
1	Regressione Lineare	0.344879	0.387655	0.391561	0.389504
2	Albero di decisione	0.235931	0.261829	0.275594	0.261532
3	Explainable Boosting Machine	0.097338	0.119824	0.125212	0.141038
4	GP for Heteroskedastic data	0.095099	0.237548	0.242309	0.226701

Risulta evidente come il metodo *StratLearn-z* rimanga imbattuto da ogni modello analizzato in questo progetto. Sia l'*Explainable Boosting Machine* che il modello *GP for Heteroskedastic data* si avvicinano alle sue prestazioni sul dataset `text_noCS`, ma lo scenario cambia del tutto con la presenza di CS. In questo caso (corrispondente agli altri tre datasets), il modello *StratLearn-z* mostra performances migliori, collocandosi stabilmente un ordine di grandezza sotto rispetto a tutti gli altri modelli analizzati. Nota a margine su *StratLearn-z*: il codice è stato testato con successo per tutti e 4 i datasets, ed i risultati sono salvati nelle varie cartelle dal nome `summary_results_X` presenti nella repo GitHub. Tra i risultati non sono riportate le metriche utilizzate poi nell'articolo, ma ci si ferma alla densità. Avendo già a disposizione i risultati delle esecuzioni in forma tabellare nell'articolo, non sono andato ad apportare modifiche al codice per eseguire il calcolo esplicito delle suddette metriche, ma mi sono limitato a controllare che il codice fosse eseguibile e i risultati salvati correttamente.

Rimane ad ogni modo il tema della explainability, punto di forza dei primi tre modelli, e non affrontato dal metodo dell'articolo.

Tutte le tecniche utilizzate hanno portato alla luce risultati tra concordanti, con la feature più importante che si è rivelata sempre essere `zy`, e le seconde due `gr` e `iz`.

Inoltre, concordanti sono risultati anche i grafici beeswarm SHAP, ed i grafici delle EBM relativi ad ogni funzione imparata.

Bibliografia

- <https://shap.readthedocs.io/en/latest/>
- <https://scikit-learn.org/stable/>
- <https://interpret.ml/docs/ebm.html>
- <https://gpflow.github.io/GPflow/2.4.0/notebooks/advanced/heteroskedastic.html>
- [arXiv:2409.20379](https://arxiv.org/abs/2409.20379)
- <https://github.com/chiamoretti/StratLearn-z>
- <https://www.aidancooper.co.uk/a-non-technical-guide-to-interpreting-shap-analyses/>